Especificação SDD — TaskAnalyzer (v1.1)

Bootcamp III, Fase 2. Autor: Eduardo Fernandes dos Santos Rocha, Ciência da Computação, matrícula 22552011.

Esta especificação é a **fonte da verdade** do projeto. Qualquer comportamento não descrito aqui não deve ser assumido nem implementado pelo agente de IA.

Histórico de versões

Versão	Descrição
1.0	Especificação original da Fase 1 (Google Docs).
1.1	Alinhamento ao enunciado da Fase 2. Detalhado abaixo.

Alterações da v1.0 para a v1.1

Item	v1.0	v1.1
Função pública	`analisar_tarefas`	`analyze_tasks(tasks: list[dict]) -> dict`
Exceção	`DadosInvalidosError` e `MetricaIndefinidaError`	`TaskValidationError` (única)
Unidade do tempo médio	minutos	horas
Chaves de saída	`quantidade_tarefas`, `total_por_status`, `indicadores_por_prioridade`	`total_tarefas`, `total_concluidas`, `total_pendentes`, `tempo_medio_conclusao_horas`, `tempo_medio_conclusao_horas_por_prioridade`, `taxa_atraso_percentual`
Lista vazia (RN-09)	levanta exceção	retorna métricas zeradas
Sem tarefas concluídas (RN-08)	levanta exceção	retorna 0.0, sem exceção

Entradas, demais regras de negócio e validações permanecem como na v1.0.

---

1. Visão geral e contrato de negócio

1.1 Propósito do módulo

O TaskAnalyzer é um módulo de análise de tarefas e produtividade. Ele recebe uma coleção de tarefas já registradas por um sistema externo e devolve métricas agregadas que permitem avaliar o desempenho de

uma equipe ou de um indivíduo em um período. O módulo não coleta, não armazena e não exibe dados: ele apenas calcula.

Objetivos funcionais:

- Calcular o tempo médio de conclusão das tarefas, no total e por prioridade.
- Calcular a taxa de atraso, entendida como o percentual de tarefas concluídas após o prazo acordado.
- Contabilizar o volume de tarefas total, concluídas e pendentes.
- Rejeitar entradas inconsistentes de forma explícita, com exceção específica e mensagem clara, em vez de produzir métricas silenciosamente incorretas.

1.2 Fora do escopo (não deve ser implementado)

- Persistência em arquivos, bancos de dados ou serviços externos.
- Interface gráfica, API HTTP ou linha de comando.
- Autenticação, autorização ou controle de acesso.
- Previsão de prazos, estimativas ou qualquer inferência estatística além das métricas descritas.

1.3 Contrato executável de interface

1.3.1 Função pública principal

```
def analyze_tasks(tasks: list[dict]) -> dict:
    """Calcula métricas de produtividade a partir de uma lista de tarefas."""
```

Assinatura única e estável. O agente de IA não pode alterar o nome nem os parâmetros desta função sem autorização explícita registrada nesta especificação.

1.3.2 Exceção do módulo

Exceção	Herda de	Quando é levantada
`TaskValidationError`	`Exception`	Entrada fora do contrato: não é lista, tarefa que não é dicionário, campo ausente, tipo incorreto, valor fora do domínio, id inválido ou duplicado, data sem fuso horário, prazo anterior à criação, ou `data_conclusao` anterior à `data_criacao`.

Exceções genéricas (`Exception`, `ValueError` puro) não devem ser levantadas diretamente ao chamador. A mensagem deve identificar o id da tarefa e o campo responsável pela falha, sempre que o id estiver disponível.

1.3.3 Entradas

O parâmetro `tasks` é uma lista de dicionários. Cada dicionário representa uma tarefa:

Campo	Tipo	Obrigatório	Descrição e restrições
`id_tarefa`	int	Sim	Identificador único e positivo (maior que zero). Não pode se repetir na lista. `bool` não é aceito.

Campo	Tipo	Obrigatório	Descrição e restrições
`titulo`	str	Sim	Texto não vazio, com no máximo 120 caracteres.
`prioridade`	str	Sim	Valores aceitos: `"baixa"`, `"media"`, `"alta"`. Comparação sem distinção de maiúsculas.
`status`	str	Sim	Valores aceitos: `"concluida"`, `"pendente"`, `"cancelada"`.
`data_criacao`	datetime	Sim	Data e hora de criação, em UTC, com fuso horário definido (timezone-aware).
`prazo`	datetime	Sim	Prazo acordado, em UTC. Deve ser posterior ou igual a `data_criacao`.
`data_conclusao`	datetime ou None	Condicional	Obrigatório quando `status` é `"concluida"`; deve ser `None` nos demais status. Deve ser posterior ou igual a `data_criacao`.

1.3.4 Saídas

A função retorna um dicionário com a estrutura fixa abaixo. Nenhuma chave adicional é permitida. Valores em ponto flutuante são arredondados para duas casas decimais.

Chave	Tipo	Descrição
`total_tarefas`	int	Total de tarefas recebidas, incluindo canceladas.
`total_concluidas`	int	Tarefas com status `"concluida"`.
`total_pendentes`	int	Tarefas com status `"pendente"`.
`tempo_medio_conclusao_horas`	float	Tempo médio, em horas, entre `data_criacao` e `data_conclusao`, considerando apenas tarefas concluídas.
`tempo_medio_conclusao_horas_por_prioridade`	dict[str, float]	Mesma média, por prioridade. As chaves `"baixa"`, `"media"` e `"alta"` estão sempre presentes.
`taxa_atraso_percentual`	float	Percentual de tarefas concluídas cuja `data_conclusao` foi posterior ao `prazo`. Varia de 0.0 a 100.0.

Exemplo da estrutura retornada:

```
{
    "total_tarefas": 5,
    "total_concluidas": 3,
    "total_pendentes": 1,
    "tempo_medio_conclusao_horas": 35.33,
    "tempo_medio_conclusao_horas_por_prioridade": {
        "baixa": 0.0,
        "media": 10.0,
        "alta": 48.0,
    },
```

```
    "taxa_atraso_percentual": 33.33,
}
```

1.3.5 Regras de negócio

Nº	Regra
RN-01	Somente tarefas com status `"concluida"` entram no cálculo de tempo médio e de taxa de atraso.
RN-02	O tempo de conclusão de uma tarefa é `(data_conclusao - data_criacao)` convertido para horas.
RN-03	Uma tarefa é atrasada quando `data_conclusao` é estritamente posterior ao `prazo`. Concluir exatamente no prazo não é atraso.
RN-04	Tarefas `"cancelada"` são ignoradas nas métricas de tempo e atraso, mas contam em `total_tarefas`.
RN-05	Todas as datas são tratadas em UTC. Datas sem fuso horário (naive) são entrada inválida.
RN-06	O arredondamento para duas casas ocorre apenas na montagem do retorno, nunca nos cálculos intermediários.
RN-07	Uma prioridade sem tarefas concluídas apresenta tempo médio igual a 0.0, sem levantar exceção.
RN-08	Quando não há nenhuma tarefa concluída, `tempo_medio_conclusao_horas` e `taxa_atraso_percentual` são 0.0, sem exceção. A divisão por zero deve ser evitada. (alterada na v1.1)
RN-09	Uma lista vazia é entrada válida e retorna todas as métricas zeradas. (alterada na v1.1)
RN-10	A ordem das tarefas na lista de entrada não influencia o resultado.
RN-11	Se a entrada não for uma lista, ou se houver `id_tarefa` repetido, levanta `TaskValidationError`. (nova na v1.1)

---

2. Cenários de aceite e Test Harness

Os cenários abaixo são parte do contrato: o agente de IA não pode alterá-los, removê-los ou adaptá-los para fazer o código passar.

2.1 Cenários de aceite (Dado, Quando, Então)

Cenário 1 (Sucesso): métricas corretas para tarefas válidas

Dado cinco tarefas válidas, todas criadas no mesmo instante, com os dados:

id	prioridade	status	prazo (h após criação)	conclusão (h após criação)
1	alta	concluida	48	24
2	alta	concluida	24	72

id	prioridade	status	prazo (h após criação)	conclusão (h após criação)
3	media	concluida	10	10
4	baixa	pendente	48	n/a
5	media	cancelada	48	n/a

Quando o analisador for executado sobre esse conjunto, **então** deve retornar `total_tarefas` 5, `total_concluidas` 3, `total_pendentes` 1, `tempo_medio_conclusao_horas` 35.33, `taxa_atraso_percentual` 33.33 e médias por prioridade `baixa` 0.0, `media` 10.0 e `alta` 48.0.

Cenário 2 (Exceção): datas inconsistentes

Dado uma tarefa concluída cuja `data_conclusao` é anterior à `data_criacao`, **quando** o analisador for executado, **então** deve levantar `TaskValidationError` com mensagem clara, sem retornar métricas parciais.

Cenário 3 (Borda): conclusão exatamente no prazo

Dado uma tarefa concluída em que `data_conclusao` é igual ao `prazo`, **quando** o analisador for executado, **então** a tarefa não deve ser contada como atrasada (RN-03).

Cenário 4 (Borda): sem tarefas concluídas

Dado uma lista vazia, ou uma lista apenas com tarefas pendentes, **quando** o analisador for executado, **então** deve retornar médias iguais a 0.0 e taxa de atraso 0.0, sem lançar exceção (RN-07, RN-08, RN-09).

Cenário 5 (Validação): campo ausente ou fora do domínio

Dado uma tarefa sem o campo `prioridade`, com `prioridade` igual a `"urgente"`, com id repetido, ou com data sem fuso horário, **quando** o analisador for executado, **então** deve levantar `TaskValidationError` identificando o id e o campo.

2.2 Planejamento do Test Harness

O Test Harness traduz os cenários em testes automatizados (pytest). Nenhum código é aceito antes de passar integralmente por ele. Se o código falhar em um teste, a correção recai sobre o código; se um teste estiver errado, a correção recai sobre esta especificação, que é revisada e versionada primeiro.

Cenário	Testes em `tests/test_harness.py`
1	`test_metricas_com_tarefas_validas`, `test_tempo_medio_por_prioridade`, `test_estrutura_do_retorno_e_fixa`, `test_ordem_das_tarefas_nao_influencia`, `test_prioridade_sem_distincao_de_caixa`
2	`test_data_conclusao_anterior_a_criacao`
3	`test_conclusao_exatamente_no_prazo`, `test_todas_concluidas_com_atraso`
4	`test_lista_vazia_retorna_zeros`, `test_apenas_pendentes_retorna_medias_zero`

Cenário	Testes em `tests/test_harness.py`
5	`test_entrada_invalida_levanta_erro` (parametrizado), `test_ids_duplicados`, `test_entrada_nao_e_lista`

Critérios do harness:

- Dados de entrada isolados em fixtures e funções auxiliares.
- Variações do mesmo comportamento usam `@pytest.mark.parametrize`.
- Datas fixas e explícitas, nunca derivadas de `datetime.now()`.
- Sem dependência de rede, arquivos, banco de dados ou ordem de execução.
- Aceite: 100% dos testes aprovados e cobertura mínima de 90% de `src/task_analyzer.py`, medida com pytest-cov.